

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное учреждение высшего образования
«Национальный исследовательский Нижегородский государственный университет им.
Н. И. Лобачевского» (ННГУ)

Институт информационных технологий, математики и механики
Кафедра математического обеспечения и суперкомпьютерных технологий
Направление подготовки: «Программная инженерия»

ОТЧЕТ

по производственной (технологической (проектно-технологической)) практике
на тему:

**«Расширение функциональности информационной системы для
музыкальной студии»**

Выполнил:

студент группы 3823Б1ПР2

_____ Клименко В. С.

(Подпись)

Научный руководитель:

кандидат ф.м.н., доцент

_____ Шапошников Д. Е.

(Подпись)

Нижний Новгород

2026

Содержание

Введение.....	4
1 Теоретическая часть (материалы и методы)	5
1.1 Назначение и архитектура веб-приложения	5
1.2 Клиентская часть веб-приложения	5
1.3 Серверная часть и REST API	5
1.4 Работа с базой данных	6
1.5 Механизм бронирования.....	6
1.6 Методы обработки дат и времени.....	7
1.7 Обеспечение корректности и надёжности работы	7
1.8 Вывод по теоретической части	7
2 Практическая часть (результаты и обсуждение)	8
2.1 Постановка задач на период практики	8
2.2 Возможность авторизации по e-mail и по номеру телефона.....	9
2.3 Доработка формы обратной связи	9
2.4 Внедрение роли администратора и административных панелей управления.....	10
2.4.1 Система аутентификации и авторизации	10
2.4.2 Админ-панель управления бронированиями (admin_bookings.js).....	10
2.4.3 Админ-панель управления инструментами (admin_instruments.js).....	11
2.4.4 Админ-панель управления помещениями (admin_rooms.js)	12
2.4.5 Админ-панель управления абонементом (admin_subscription_plans.js).....	12
2.4.6 Интеграция административных функций в клиентскую часть.....	12
2.4.7 Технические аспекты реализации.....	13
2.4.8 Результаты внедрения	13
2.5 Доработка логики статусов бронирования в базе данных.....	13
2.5.1 Постановка задачи.....	13
2.5.2 Реализованное решение	14
2.5.3 Результаты внедрения	14
2.6 Синхронизация даты перенесённого бронирования с датой в профиле пользователя.....	15
2.6.1 Постановка задачи.....	15
2.6.2 Причина возникновения проблемы	15
2.6.3 Реализованное решение	15
2.6.4 Результаты внедрения	16
2.7 Расширение функционала для роли администратора	16

2.7.1 Удобное окно для переноса бронирования	16
2.7.2 Редактирование каталога инструментов	17
2.7.3 Редактирование каталога помещений	18
2.7.4 Редактирование прайс-листа	19
2.8 Реализация функционала счетов	19
2.8.1 Постановка задачи.....	19
2.8.2 Реализованное решение	19
2.9 Добавление дополнительных инструментов при бронировании помещений	20
2.9.1 Постановка задачи.....	20
2.9.2 Реализованное решение	20
2.9.3 Результаты внедрения	21
2.10 Поиск по дате в каталогах	21
2.10.1 Постановка задачи.....	21
2.10.2 Реализованное решение	22
2.10.3 Результаты внедрения	22
2.11 Функционал абонементов.....	22
2.11.1 Постановка задачи.....	22
2.11.2 Реализованное решение	23
2.11.3 Результаты внедрения	24
2.12 Внедрение уведомлений о бронированиях	24
2.12.1 Постановка задачи.....	24
2.12.2 Реализованное решение	24
2.12.3 Результаты внедрения	26
2.13 Завершение интеграции с базой данных.....	26
2.13.1 Постановка задачи.....	26
2.13.2 Реализованное решение	26
2.13.3 Результаты внедрения	27
2.14 Улучшение пользовательского интерфейса	27
2.14.1 Постановка задачи.....	27
2.14.2 Реализованное решение	28
2.14.3 Результаты внедрения	28
Заключение	29
Список литературы	30

Введение

В условиях цифровизации сервисов аренды и бронирования всё большее значение приобретают веб-приложения, обеспечивающие удобный доступ пользователей к услугам в режиме реального времени. Актуальной также является автоматизация процессов аренды музыкального оборудования и помещений студии звукозаписи, где важны наглядность каталога, прозрачность стоимости и корректная фиксация бронирований.

В ходе поиска информации по теме и подбора концепции было выяснено, что решения, пересекающиеся по функционалу с разрабатываемой мной информационной системой, существуют, но не все в полной мере удовлетворяют требованиям музыкальной студии. Одна часть решений ориентирована на расписание занятий и управление студентами, меньше внимания уделяя другим рабочим процессам музыкальной студии. Другая часть довольно сложна в настройке и либо имеет перегруженный интерфейс, либо является консольными приложениями, что не совсем удобно в использовании, в т.ч. для управления процессами. Есть и третья вариация решений - учебные и Open-Source проекты, которые могут иметь широкий функционал, но так и не доходить до внедрения.

Целью данной работы является создание нового и дополнительного функционала веб-приложения для аренды музыкальных инструментов и помещений, предоставляющего пользователю возможность просматривать каталоги, получать подробную информацию о необходимой ему категории, выбирать сроки аренды и оформлять бронирование с последующим сохранением данных в базе данных.

Преимуществом моего приложения является объединение трёх важных направлений для современной студии: управления ресурсами, бронирования и бизнес-логики, что делает данную реализацию более гибкой и универсальной. При этом система будет иметь лёгкий и визуально привлекательный интерфейс, что делает работу с ней более комфортной для конечного пользователя. Моё решение можно будет легко адаптировать под любое изменение в бизнес-процессах студии.

В процессе разработки были использованы современные веб-технологии: клиентская часть реализована с применением языка гипертекстовой разметки HTML, за работу со стилями отвечает CSS, а в качестве языка программирования выступает JavaScript как наиболее часто используемый в веб-разработке. Взаимодействие с сервером осуществляется через REST API. Серверная часть реализована на платформе ASP.NET с использованием базы данных для хранения информации об инструментах и бронированиях. Такой подход обеспечивает расширяемость, модульность и удобство сопровождения системы.

1 Теоретическая часть (материалы и методы)

1.1 Назначение и архитектура веб-приложения

Разрабатываемое веб-приложение предназначено для автоматизации процесса аренды музыкальных инструментов и управления бронированиями помещениями музыкальной студии. Приложение реализовано в виде клиент-серверной системы, что позволяет разделить ответственность между пользовательским интерфейсом и серверной логикой, а также обеспечить масштабируемость.

Архитектура приложения построена по принципу клиент–серверного взаимодействия. Клиентская часть отвечает за отображение данных, взаимодействие с пользователем и предварительную обработку информации. Серверная часть предоставляет REST API, осуществляет бизнес-логику и обеспечивает доступ к базе данных.

Такой подход позволяет централизованно управлять данными, обеспечить их целостность и реализовать единый механизм обработки запросов от различных клиентов.

1.2 Клиентская часть веб-приложения

Клиентская часть приложения реализована с использованием стандартных веб-технологий: HTML, CSS и JavaScript. HTML используется для формирования структуры страниц, CSS — для визуального оформления и адаптивного отображения элементов интерфейса, JavaScript — для реализации интерактивной логики и работы с данными.

JavaScript применяется для:

- обработки событий пользовательского интерфейса;
- динамического обновления содержимого страниц без перезагрузки;
- загрузки данных с сервера с использованием асинхронных HTTP-запросов;
- расчёта стоимости аренды на основе выбранных дат;
- валидации пользовательских данных.

Для взаимодействия с сервером используется встроенный API fetch, позволяющий отправлять HTTP-запросы и обрабатывать ответы в формате JSON. Асинхронный подход обеспечивает отзывчивость интерфейса и улучшает пользовательский опыт.

1.3 Серверная часть и REST API

Серверная часть приложения реализована на платформе ASP.NET и предоставляет REST API для доступа к данным. Использование архитектурного стиля REST позволяет

организовать взаимодействие между клиентом и сервером с применением стандартных HTTP-методов (GET, POST, PUT, DELETE).

Основные функции серверной части включают:

- предоставление данных о музыкальных инструментах;
- предоставление данных о помещениях;
- обработку запросов на создание бронирований;
- валидацию входных данных;
- сохранение информации в базе данных.

Передача данных между клиентом и сервером осуществляется в формате JSON, что обеспечивает совместимость и удобство обработки информации на обеих сторонах.

1.4 Работа с базой данных

Для хранения информации используется реляционная база данных, в которой размещаются данные об инструментах, пользователях и бронированиях. База данных развёрнута в СУБД PostgreSQL. Каждое бронирование содержит сведения о пользователе, выбранном инструменте, сроках аренды и текущем статусе.

Связь между сущностями реализуется с использованием первичных и внешних ключей, что обеспечивает целостность данных и предотвращает появление некорректных записей. Доступ к базе данных осуществляется через серверную часть приложения, что исключает прямое взаимодействие клиента с хранилищем данных и повышает уровень безопасности.

1.5 Механизм бронирования

Процесс бронирования инструмента реализован в несколько этапов. На первом этапе пользователь выбирает инструмент и задаёт период аренды. На клиентской стороне производится расчёт итоговой стоимости на основе количества дней аренды и стоимости одного дня.

На следующем этапе пользователь подтверждает бронирование. После подтверждения данные отправляются на сервер посредством HTTP-запроса методом POST. Сервер обрабатывает запрос, сохраняет информацию о бронировании в базе данных и возвращает результат выполнения операции.

Для временного хранения данных бронирования на клиентской стороне используются механизмы `localStorage` и `sessionStorage`, что позволяет сохранять состояние между страницами без необходимости повторного обращения к серверу.

1.6 Методы обработки дат и времени

В приложении используются методы работы с датами для корректного определения сроков аренды и расчёта стоимости. Для бронирования инструментов пользователь выбирает даты начала и окончания аренды, после чего вычисляется количество дней аренды. В случае, если пользователь решит бронировать помещение, ему необходимо выбрать день, время и количество часов, на которое оно будет забронировано.

Для обеспечения согласованности данных применяется преобразование локального времени пользователя в единый формат (ISO 8601) с учётом временной зоны. Это позволяет корректно сохранять и обрабатывать временные данные на серверной стороне.

1.7 Обеспечение корректности и надёжности работы

Для повышения надёжности приложения реализована обработка ошибок, возникающих при загрузке данных или выполнении сетевых запросов. В случае возникновения ошибок пользователю отображаются соответствующие уведомления в консоли Web API, а также в консоли разработчика в браузере (тестирование работоспособности веб-приложения и корректного отображения страниц происходило в Яндекс Браузере и в Microsoft Edge).

Также предусмотрена проверка авторизации пользователя перед подтверждением бронирования, что предотвращает создание заказов неавторизованными пользователями. Если неавторизованный пользователь делает попытку бронирования, система предлагает выдаёт сообщение о необходимости входа и переходит на соответствующую страницу.

1.8 Вывод по теоретической части

В ходе изучения теоретических основ были рассмотрены принципы построения клиент-серверных веб-приложений, методы организации REST API, а также способы работы с базами данных и пользовательским интерфейсом. Полученные знания были использованы при разработке веб-приложения аренды музыкальных инструментов и студийных помещений и легли в основу практической реализации проекта.

2 Практическая часть (результаты и обсуждение)

2.1 Постановка задач на период практики

В этом семестре мной и другими студентами-практикантами совместно с руководителем было принято решение создать документ для управления задачами. В данном документе мы составили список текущих недочётов и желаемых нововведений, распределённый по категориям и приоритету, и расставили ориентировочное время выполнения для каждого из них, чтобы впоследствии сравнивать с реальной датой завершения работы над пунктом списка.

Среди задач для моего проекта были выделены следующие:

- возможность авторизации не только по адресу электронной почты, но и по номеру телефона;
- доработка формы обратной связи;
- внедрение роли администратора и административных панелей управления в систему (для бронирований, инструментов, помещений и абонементов соответственно);
- доработка логики статусов бронирования в базе данных: для каждого бронирования в таблице базы данных ранее был указан статус (in progress, completed, cancelled), но иногда для отменённого прошедшего по времени бронирования статус не менялся, так и оставаясь in progress;
- синхронизация даты перенесённого администратором бронирования с датой в профиле пользователя, чьё бронирование перенесено;
- расширение функционала для роли администратора: добавление удобных модальных окон для выбора новой даты и времени бронирования, возможности редактирования каталогов инструментов и помещений, а также страницы с ценами;
- реализация функционала счетов: в таблице оставались две незадействованные таблицы bills_advanced и bill_items_advanced, в первой таблице должна быть отражена сумма бронирования и статус оплаты, во второй – непосредственно пункты бронирования;
- добавление дополнительных инструментов, которые можно выбрать при бронировании помещения, если таких первоначально нет в выбранном помещении, увеличит общую стоимость бронирования;

- поиск по дате в каталогах: пользователю нужна возможность видеть доступные в нужные ему дату и время в каталоге, что нагляднее и быстрее, чем заходить за этой информацией в карточку инструмента/помещения;
- функционал абонементов: абонементы были реализованы, но не использовались и не давали пользователю преференций при бронированиях, необходимо написать логику их работы, согласно которой абонемент даёт скидку на инструмент/помещение и расходует количество доступных по нему сеансов, оба показателя зависят от выбранного тарифа;
- внедрение уведомлений о бронированиях: пользователь должен видеть в интерфейсе, что бронирование совершено или перенесено/отменено администратором, иметь возможность пометить одно или несколько уведомлений как прочитанные;
- полная интеграция с базой данных: каждая таблица БД должна быть задействована при работе информационной системы, нужно дописать логику для упомянутых ранее дополнительных инструментов и счетов;
- улучшение пользовательского интерфейса после выполнения перечисленных задач.

2.2 Возможность авторизации по e-mail и по номеру телефона

На странице входа в систему были добавлены чекбоксы (флажки), чтобы пользователь мог выбрать контакт, с помощью которого он хочет авторизоваться: адрес электронной почты или свой номер телефона. Каждый из этих пунктов указывается при регистрации в системе и сохраняется в базу данных вместе с фамилией и именем пользователя. В финальной версии чекбоксы заменены на слайдер для более современного внешнего вида интерфейса.

Появление выбора между этими двумя контактами делает упрощает авторизацию, т.к. пользователь может не помнить наизусть какой-либо из них или использовать новый номер/адрес электронной почты, которого нет в базе данных.

2.3 Доработка формы обратной связи

В первой версии страницы контактов была реализована форма обратной связи, в которой можно было указать имя адресанта, адрес электронной почты и текст сообщения для отправки на имя представителей музыкальной студии. После анализа прочих элементов этой страницы я посчитал наличие формы обратной связи избыточным, поскольку на странице уже указан ряд контактов для взаимодействия (электронная почта, страница в социальной сети VK,

профиль GitHub и канал на YouTube). Пользователь может использовать каждый из них в зависимости от того, какой ему доступен или более удобен.

Однако удачной идеей показалось добавить секцию FAQ (frequently asked questions, часто задаваемые вопросы), где были бы указаны пояснения к моментам, потенциально интересующим пользователя. Следовательно, форма обратной связи оказалась заменена на FAQ-секцию.

2.4 Внедрение роли администратора и административных панелей управления

2.4.1 Система аутентификации и авторизации

В рамках реализации ролевой модели доступа была разработана система авторизации, которая при входе пользователя проверяет его принадлежность к категории администраторов. При успешной аутентификации система выполняет дополнительный запрос к эндпоинту `StaffAdvanced/byUser/{userId}` для определения наличия у пользователя административных прав. В зависимости от результата проверки в `localStorage` устанавливается флаг `isStaff`, который используется для разграничения доступа на клиентской стороне.

Была реализована динамическая адаптация пользовательского интерфейса:

- для администраторов отображаются дополнительные элементы управления на страницах каталогов;
- обычным пользователям скрываются административные функции;
- меню навигации корректно отображается в зависимости от статуса авторизации.

2.4.2 Админ-панель управления бронированиями (admin_bookings.js)

При загрузке страницы выполняется проверка:

- наличие авторизации пользователя в системе;
- наличие прав администратора через API-запрос;
- при отсутствии прав выполняется редирект на страницу входа.

Реализован механизм автоматического обновления статусов бронирований при загрузке страницы. Система отправляет POST-запрос к эндпоинту `admin/refresh-expired-statuses`, который переводит просроченные бронирования из статуса "in progress" в статус "completed".

Разработана таблица для просмотра всех бронирований с информацией:

- номер заказа (первые 4 символа UUID);
- данные пользователя (ФИО, email);

- наименование забронированного ресурса (помещение или инструмент);
- время начала и окончания бронирования;
- текущий статус (in progress, completed, cancelled);
- ответственный администратор.

Реализована возможность отмены бронирования администратором:

- кнопка доступна только для активных бронирований;
- подтверждение действия через диалоговое окно;
- отправка PUT-запроса к эндпоинту BookingsAdvanced/{bookingId}/cancel;
- автоматическое обновление страницы после успешной операции.

Разработан механизм переноса бронирования на новое время:

- выбор новой даты и времени начала через календарь;
- выбор новой даты и времени окончания;
- валидация корректности временного интервала (начало<окончание);
- отправка PUT-запроса к эндпоинту BookingsAdvanced/{bookingId}/reschedule.

2.4.3 Админ-панель управления инструментами (admin_instruments.js)

Разработана таблица для управления каталогом инструментов с возможностью:

- просмотра всех инструментов с их характеристиками (ID, название, категория, цена, состояние);
- редактирования существующих инструментов;
- удаления инструментов из каталога.

Реализована форма добавления инструмента с полями:

- название инструмента;
- категория (выбор из предустановленных);
- цена аренды;
- описание;
- ссылка на изображение;
- цвет;
- состояние (отличное, хорошее, незначительные дефекты, в ремонте);
- ориентация (для гитар: правша/левша/неприменимо);
- флаг доступности для аренды.

Разработан механизм редактирования с модальным окном:

- загрузка актуальных данных инструмента;

- предзаполнение формы существующими значениями;
- отправка PUT-запроса для обновления информации.

2.4.4 Админ-панель управления помещениями (admin_rooms.js)

Реализован полный CRUD-функционал для управления помещениями:

- просмотр списка всех помещений;
- добавление новых помещений;
- редактирование существующих;
- удаление помещений из каталога.

При управлении помещениями администратор может настраивать:

- название помещения;
- тип помещения (лаунж-зона/студия звукозаписи/репетиционный зал);
- статус доступности (свободно/занято).

2.4.5 Админ-панель управления абонементом (admin_subscription_plans.js)

Разработан функционал для управления абонементом:

- просмотр списка всех тарифных планов;
- добавление новых тарифов;
- редактирование существующих;
- удаление тарифов.

При создании или редактировании абонемента администратор может настроить:

- название тарифного плана;
- общую стоимость;
- срок действия (в днях);
- количество доступных сессий;
- описание тарифа.

2.4.6 Интеграция административных функций в клиентскую часть

На страницах каталогов инструментов и помещений для авторизованных администраторов отображается кнопка "Добавить инструмент/помещение". При нажатии открывается модальное окно с формой добавления нового элемента.

Для администраторов на карточках инструментов и помещений отображаются кнопки удаления (крестик). При нажатии выполняется проверка через подтверждающий диалог и последующее удаление элемента через API.

2.4.7 Технические аспекты реализации

API-эндпоинты, использованные в административной части:

- StaffAdvanced/byUser/{userId} — проверка административных прав;
- BookingsAdvanced/admin — получение всех бронирований;
- BookingsAdvanced/admin/refresh-expired-statuses — обновление просроченных статусов;
- BookingsAdvanced/{bookingId}/cancel — отмена бронирования;
- BookingsAdvanced/{bookingId}/reschedule — перенос бронирования;
- Equipments — CRUD-операции с инструментами;
- Rooms — CRUD-операции с помещениями;
- SubscriptionPlans — CRUD-операции с абонементом.

Механизмы безопасности:

- проверка прав администратора перед загрузкой каждой административной страницы;
- передача staffUserId в теле запросов для аудита действий;
- отсутствие административных кнопок в интерфейсе обычных пользователей.

2.4.8 Результаты внедрения

В результате проведённой работы были созданы полнофункциональные административные панели, позволяющие:

- управлять всеми бронированиями в системе (просмотр, отмена, перенос);
- поддерживать актуальность каталога инструментов (добавление, редактирование, удаление);
- управлять каталогом помещений;
- настраивать тарифные планы абонементов;
- обеспечить разграничение прав между обычными пользователями и администраторами.

2.5 Доработка логики статусов бронирования в базе данных

2.5.1 Постановка задачи

В процессе эксплуатации информационной системы была выявлена логическая несогласованность в управлении статусами бронирований. Для каждого бронирования в таблице базы данных предусмотрены следующие статусы:

- in progress — активное бронирование, период действия которого ещё не завершён;
- completed — бронирование, период действия которого успешно завершён;
- cancelled — бронирование, отменённое пользователем или администратором до наступления времени начала.

Однако в исходной реализации отсутствовал механизм автоматического обновления статусов бронирований по истечении их временного периода. Это приводило к ситуации, когда бронирование, время окончания которого уже наступило, продолжало числиться в статусе "in progress", что искажало статистику и создавало неудобства при администрировании.

2.5.2 Реализованное решение

Для устранения данной проблемы был разработан и внедрён механизм автоматического обновления статусов бронирований при загрузке административной панели.

Создан дополнительный эндпоинт на стороне сервера:

Маршрут: POST /api/BookingsAdvanced/admin/refresh-expired-statuses

Назначение: поиск и обновление всех просроченных бронирований, имеющих статус "in progress"

На стороне клиента (административная панель) реализован вызов данного эндпоинта.

Логика работы механизма:

- при загрузке страницы административного управления бронированиями автоматически выполняется запрос на обновление статусов;
- сервер проверяет все бронирования со статусом "in progress" и сравнивает время их окончания с текущей датой и временем;
- для бронирований, у которых время окончания меньше текущего момента, статус автоматически изменяется на "completed";
- в запросе также передаётся идентификатор администратора, что позволяет вести аудит действий в системе.

2.5.3 Результаты внедрения

После реализации данного решения были получены следующие результаты:

- автоматизация процесса: исключена необходимость ручного перевода бронирований в статус "completed" после их фактического завершения;

- актуальность данных: статусы бронирований в любой момент времени соответствуют их реальному состоянию;
- улучшение пользовательского опыта: администратор видит корректный список активных и завершённых бронирований без необходимости выполнять дополнительные действия;
- упрощение аналитики: на основе корректных статусов можно строить достоверные отчёты по загруженности помещений и инструментов.

2.6 Синхронизация даты перенесённого бронирования с датой в профиле пользователя

2.6.1 Постановка задачи

В процессе эксплуатации информационной системы была выявлена проблема согласованности данных при переносе бронирований администратором. Исходная архитектура системы предполагала, что после выполнения администратором операции переноса (reschedule) бронирования на новую дату и время, пользователь, просматривая свои бронирования в личном кабинете, продолжает видеть старые, неактуальные даты. Это создавало путаницу и могло приводить к пропуску пользователем актуального времени бронирования.

2.6.2 Причина возникновения проблемы

Проблема была обусловлена архитектурным решением, при котором на стороне клиента использовались два независимых механизма хранения данных о бронированиях:

- API-запросы к серверу — для получения актуальных бронирований конкретного пользователя через эндпоинт `BookingsAdvanced/userHistory/{userId}`;
- локальное хранилище (localStorage) — для кэширования истории бронирований с целью обеспечения работы приложения при отсутствии соединения с сервером.

При переносе бронирования администратором обновление происходило только в основной базе данных, но не в локальном хранилище пользователя, что приводило к рассинхронизации данных.

2.6.3 Реализованное решение

Для устранения данной проблемы был реализован комплексный подход, гарантирующий актуальность отображаемых данных в профиле пользователя:

1. Принудительное обновление данных при загрузке страницы профиля.

В функции `loadBookingsFromAPI` реализован механизм принудительной загрузки актуальных данных из API при каждом переходе пользователя на страницу профиля.

2. Автоматическое обновление `localStorage` после успешной операции переноса.

При успешном выполнении переноса бронирования администратором на стороне клиента выполняется обновление локального хранилища через повторный вызов функции загрузки.

3. Механизм обогащения данных (`enrichBookingsWithDetails`).

Для корректного отображения информации о бронировании после переноса была реализована функция `enrichBookingsWithDetails`, которая:

- загружает актуальные справочные данные (инструменты, помещения, типы помещений);
- формирует единый объект бронирования с полными данными для отображения;
- определяет актуальный статус бронирования на основе временных меток.

2.6.4 Результаты внедрения

После реализации данного решения были получены следующие результаты:

- согласованность данных: пользователь всегда видит актуальную дату и время бронирования после любого переноса, выполненного администратором;
- прозрачность операций: отсутствие расхождений между данными в API и отображаемыми в интерфейсе;
- устойчивость к сетевым сбоям: при недоступности сервера система использует локально сохранённые данные с соответствующим уведомлением пользователя;
- единый источник истины: API является единственным авторитетным источником данных о бронированиях, локальное хранилище используется только как кэш.

2.7 Расширение функционала для роли администратора

В рамках развития административной подсистемы был значительно расширен функционал, доступный пользователям с ролью администратора. Реализованные возможности позволяют эффективно управлять ключевыми аспектами работы музыкальной студии: бронированиями, каталогами инструментов и помещений, а также ценообразованием.

2.7.1 Удобное окно для переноса бронирования

В процессе эксплуатации системы была выявлена потребность в гибком управлении временем бронирований. Клиенты студии могут обращаться с просьбой перенести время

аренды, что требует от администратора возможности быстро и удобно изменить дату и время начала/окончания бронирования.

Разработан удобный интерфейс для переноса бронирований, интегрированный непосредственно в таблицу управления бронированиями.

Ключевые элементы реализации:

- кнопка "Перенести" — для каждого активного бронирования (со статусом "in progress") в административной панели отображается кнопка переноса, для завершённых или отменённых бронирований данная кнопка недоступна, что предотвращает некорректные операции;
- модальное окно выбора даты и времени — при нажатии на кнопку переноса открывается кастомное модальное окно, позволяющее администратору выбрать новую дату и время, окно реализовано с использованием собственного компонента `showDateTimePicker`, который обеспечивает единообразный интерфейс для выбора временных параметров.

Пошаговый процесс переноса:

- администратор нажимает кнопку "Перенести" для выбранного бронирования;
- открывается модальное окно для выбора новой даты и времени начала;
- после подтверждения открывается окно для выбора новой даты и времени окончания;
- выполняется валидация введённых данных (дата начала должна быть раньше даты окончания);
- при успешной валидации отправляется PUT-запрос на сервер для обновления временных параметров бронирования;
- обратная связь — после успешного переноса пользователь получает подтверждение через alert-сообщение, а страница автоматически обновляется для отображения актуальных данных.

2.7.2 Редактирование каталога инструментов

Реализована полноценная панель для управления каталогом инструментов, доступная только авторизованным администраторам.

Реализованные функции:

- просмотр всех инструментов: табличное отображение с колонками: ID, название, категория, цена, состояние, действия;

- добавление инструмента: модальное окно с формой, содержащей все необходимые поля (название, категория, цена, описание, изображение, цвет, состояние, ориентация, доступность);
- редактирование инструмента: возможность изменения всех параметров существующего инструмента через модальное окно с предзаполненными данными;
- удаление инструмента: с подтверждением действия через диалоговое окно.

Для удобства администратора функции добавления и удаления инструментов продублированы непосредственно на странице каталога инструментов:

- кнопка "Добавить инструмент" — отображается в интерфейсе каталога только для администраторов, при нажатии открывается модальное окно с формой добавления нового инструмента;
- кнопка удаления (крестик) на карточке инструмента — для каждого инструмента в каталоге администратор видит кнопку удаления. При нажатии выполняется проверка через confirm-диалог, после чего отправляется DELETE-запрос на сервер, и карточка инструмента удаляется из DOM.

2.7.3 Редактирование каталога помещений

Реализована панель управления помещениями с полным CRUD-функционалом:

- просмотр помещений: таблица с колонками: ID, название, тип, статус доступности, действия;
- добавление помещения: форма с полями: название, тип помещения (Лаунж-зона / Студия звукозаписи / Репетиционный зал), флаг доступности;
- редактирование помещения: изменение всех параметров существующего помещения;
- удаление помещения: с подтверждением действия.

Так же, как и у инструментов, функции управления помещениями доступны непосредственно на странице каталога:

- кнопка "Добавить помещение" — реализована в виде отдельной карточки для внешнего удобства и соответствия современным тенденциям в разработке пользовательского интерфейса, появляется в интерфейсе для администраторов, открывает форму добавления нового помещения;
- кнопка удаления (крестик) на карточке помещения — позволяет администратору быстро удалить помещение из каталога.

2.7.4 Редактирование прайс-листа

Реализована панель для управления тарифными планами абонементов, позволяющая администратору гибко настраивать ценообразование:

- просмотр абонементов: таблица с информацией о всех тарифных планах;
- добавление абонемента: форма с параметрами: название, цена, срок действия (дней), количество сессий, описание;
- редактирование абонемента: изменение параметров существующего тарифного плана;
- удаление абонемента: удаление тарифа с подтверждением.

Для управления отображением цен на клиентской части реализован механизм кастомных карточек цен:

- кнопка "Редактировать цены" — отображается на странице `prices.html` для администраторов, при нажатии переводит страницу в режим редактирования;
- режим редактирования: все ценовые карточки становятся редактируемыми (название, цена, единица измерения); появляется возможность добавлять новые кастомные карточки; для кастомных карточек доступно удаление; кнопка "Редактировать цены" меняет текст на "Сохранить изменения";
- сохранение изменений: все изменения сохраняются в `localStorage` под ключом `customPriceCards` и подгружаются при последующих загрузках страницы, переопределяя базовые цены из API.

2.8 Реализация функционала счетов

2.8.1 Постановка задачи

В процессе развития информационной системы музыкальной студии возникла необходимость внедрения подсистемы учёта финансовых операций. На момент начала работ в базе данных уже существовали таблицы `bills_advanced` и `bill_items_advanced`, однако они не использовались в бизнес-логике приложения. Требовалось реализовать механизм автоматического формирования счетов на основании подтверждённых бронирований с возможностью отслеживания статуса оплаты и детализации услуг.

2.8.2 Реализованное решение

В рамках реализации были задействованы следующие сущности:

- BillsAdvanced: хранит информацию о счете: уникальный идентификатор, ссылку на бронирование, пользователя, итоговую сумму, статус оплаты, флаг использования абонемента, дату создания;
- BillItemsAdvanced: хранит детализацию счёта: тип услуги, наименование, количество, цену за единицу, итоговую стоимость позиции.

В контроллере BookingsAdvancedController при обработке POST-запроса на создание бронирования была реализована логика формирования счёта. Для обеспечения прозрачности финансовых операций для пользователя реализовано создание отдельных позиций счёта. После расчёта всех позиций обновляется итоговая сумма счёта и сохраняются все записи.

2.9 Добавление дополнительных инструментов при бронировании помещений

2.9.1 Постановка задачи

В процессе развития функционала информационной системы музыкальной студии была выявлена потребность в расширении возможностей бронирования помещений. Исходная реализация позволяла арендовать только непосредственно помещение без возможности добавления сопутствующего оборудования. Однако в реальных условиях работы студии клиенты часто нуждаются не только в помещении, но и в дополнительных инструментах (микрофоны, синтезаторы и т.д.), которые отсутствуют в выбранном помещении на постоянной основе.

Требования к функционалу:

- возможность выбора дополнительного оборудования при бронировании помещения;
- отображение стоимости дополнительного оборудования в реальном времени;
- автоматическое увеличение общей стоимости бронирования с учётом выбранных инструментов;
- корректное отображение выбранного оборудования в истории бронирований.

2.9.2 Реализованное решение

Для реализации функционала была задействована существующая таблица BookingEquipments, обеспечивающая связь многие-ко-многим между бронированиями и оборудованием: BookingsAdvanced $\longleftrightarrow$ BookingEquipments $\longleftrightarrow$ Equipment. Equipment хранит информацию об инструментах (название, цена, категория), BookingEquipments связывает бронирование с выбранным оборудованием.

В контроллере `BookingsAdvancedController` в класс `CreateBookingRequest` было добавлено поле для передачи выбранного оборудования. При создании бронирования после сохранения основных данных выполняется добавление записей о выбранном оборудовании. При формировании счёта (`BillsAdvanced` и `BillItemsAdvanced`) учитывается стоимость каждого выбранного инструмента. Для отображения выбранного оборудования в профиле пользователя в эндпоинт `GetUserBookingsHistory` был добавлен массив идентификаторов оборудования.

На странице детального просмотра помещения (`room.html`) реализована динамическая загрузка списка доступного оборудования. Для каждого доступного инструмента отображается чекбокс с указанием цены. При изменении выбора оборудования выполняется пересчёт итоговой суммы. При нажатии кнопки "Забронировать" собираются идентификаторы выбранного оборудования. В профиле пользователя (`profile.js`) выбранное оборудование отображается в составе информации о бронировании.

2.9.3 Результаты внедрения

После реализации данного решения были получены следующие результаты:

- возможность аренды дополнительного оборудования доступна;
- динамический пересчёт стоимости в реальном времени;
- фиксация оборудования в истории бронирований отображается с ценами;
- учёт скидки на дополнительное оборудование автоматически применяется.

2.10 Поиск по дате в каталогах

2.10.1 Постановка задачи

В процессе анализа пользовательского опыта была выявлена проблема: для проверки доступности инструмента или помещения на конкретную дату пользователю требовалось переходить на страницу детального просмотра каждого элемента. Это создавало неудобства при поиске свободных ресурсов, особенно когда пользователю нужно было найти любой доступный вариант среди множества.

Требования к функционалу:

- возможность фильтрации каталога инструментов и помещений по выбранной дате;
- отображение только тех элементов, которые свободны в указанную дату;
- для помещений — дополнительная фильтрация по временному интервалу;
- визуальная индикация активного фильтра с возможностью его быстрой очистки.

2.10.2 Реализованное решение

Для обеспечения функционала фильтрации по дате были разработаны следующие API-эндпоинты:

- GET /api/BookingsAdvanced/available-ids?date={date}&type=instrument - возвращает идентификаторы инструментов, свободных в указанную дату;
- GET /api/BookingsAdvanced/available-room-slots?date={date}&startTime={time}&endTime={time} - идентификаторы помещений, свободных в указанный временной интервал.

На страницах каталогов был добавлен блок фильтрации по дате для инструментов и для помещений. Фильтр по дате интегрирован в общую функцию applyFilters(), что позволяет комбинировать его с другими критериями (категория, цена, состояние и т.д.). Для информирования пользователя о применённом фильтре отображается специальное сообщение. Для помещений реализована более точная фильтрация с учётом временного интервала.

2.10.3 Результаты внедрения

После реализации данного решения были получены следующие результаты:

- низкое время поиска свободного ресурса (фильтрация в каталоге);
- возможность комбинирования с другими фильтрами доступна;
- фильтрация помещений по временному интервал доступна;
- визуальная обратная связь о применённом фильтре отображается;
- запрет выбора прошедшей даты реализован.

2.11 Функционал абонементов

2.11.1 Постановка задачи

В процессе анализа существующей информационной системы было выявлено, что модуль абонементов находился в нерабочем состоянии. Несмотря на наличие в базе данных таблиц SubscriptionPlans (тарифные планы) и UserSubscriptionsAdvanced (приобретённые пользователем абонементы), соответствующий функционал не был интегрирован в бизнес-логику бронирования. Пользователи могли приобретать абонементы, но при создании бронирования система не учитывала их наличие, не применяла скидку и не списывала сеансы.

Требования к реализации:

- при создании бронирования проверять наличие у пользователя активного абонемента;
- автоматически применять скидку на бронирование в соответствии с выбранным тарифом;
- списывать один сеанс при каждом бронировании;
- деактивировать абонемент при исчерпании количества сеансов или истечении срока действия.

2.11.2 Реализованное решение

В качестве моделей данных используются следующие таблицы БД:

- SubscriptionPlans - тарифные планы с параметрами: название, количество сеансов, стоимость, срок действия, процент скидки;
- UserSubscriptionsAdvanced - приобретённые пользователем абонементы: статус активности, оставшееся количество сеансов, дата окончания действия;

В контроллере BookingsAdvancedController при создании бронирования реализована проверка наличия у пользователя активного абонемента. Условия активного абонемента:

- IsActive == true — абонемент не отменён;
- SessionsRemaining > 0 — остались неиспользованные сеансы;
- ValidUntil > DateTime.Now — срок действия не истёк.

При наличии активного абонемента из соответствующего тарифного плана извлекается процент скидки и рассчитывается коэффициент для применения к стоимости бронирования. Коэффициент скидки применяется ко всем позициям счёта. Информация о применении абонемента сохраняется в таблице счетов для последующего отображения пользователю. Для поддержки функционала скидок в модель тарифного плана SubscriptionPlan было добавлено поле DiscountPercentage, соответствующее полю discount_percentage в таблице БД. Это поле хранит размер скидки в процентах, предоставляемой абонементом.

На странице профиля реализовано отображение активного абонемента с указанием:

- названия тарифного плана;
- оставшегося количества сеансов;
- даты окончания действия;
- размера скидки.

В карточках инструментов и помещений при наличии скидки отображается зачёркнутая исходная цена и итоговая со скидкой. Реализован функционал отмены подписки через отдельный эндпоинт.

2.11.3 Результаты внедрения

После реализации данного решения были получены следующие результаты:

- проверка абонеента при бронировании выполняется;
- применение скидки происходит автоматически при наличии абонеента;
- списание сеансов при бронировании по абонеенту выполняется;
- отображение скидки в каталогах отображается;
- отмена подписки реализована;
- автоматическая деактивация при исчерпании выполняется.

2.12 Внедрение уведомлений о бронированиях

2.12.1 Постановка задачи

В процессе эксплуатации информационной системы музыкальной студии была выявлена потребность в информировании пользователей о событиях, связанных с их бронированиями. Пользователи не имели возможности оперативно узнавать о подтверждении бронирования, его переносе или отмене администратором, что создавало неудобства и могло приводить к пропуску актуальной информации о времени аренды.

Требования к функционалу:

- автоматическое уведомление пользователя при создании бронирования;
- уведомление при переносе бронирования администратором;
- уведомление при отмене бронирования администратором;
- визуальное отображение непрочитанных уведомлений;
- возможность отметить одно уведомление как прочитанное;
- возможность отметить все уведомления как прочитанные.

2.12.2 Реализованное решение

Для хранения уведомлений была создана таблица Notification со следующей структурой:

- NotificationId (GUID) - уникальный идентификатор уведомления;
- UserId (GUID) - идентификатор пользователя-получателя;

- BookingUid (GUID) - ссылка на связанное бронирование;
- NotificationType (string) - тип уведомления (new_booking, booking_rescheduled, booking_cancelled);
- Title (string) - заголовок уведомления;
- Message (string) - текст уведомления;
- CreatedAt (DateTime) - дата и время создания;
- IsRead (bool) - флаг прочтения.

В контроллере NotificationsController были реализованы следующие эндпоинты:

- GET /api/Notifications/user/{userId} - получение всех уведомлений пользователя;
- GET /api/Notifications/user/{userId}/unread - получение количества непрочитанных уведомлений;
- PUT /api/Notifications/{notificationId}/read - отметка одного уведомления как прочитанного;
- PUT /api/Notifications/user/{userId}/mark-all-read - отметка всех уведомлений как прочитанных;
- POST /api/Notifications - создание уведомления (для внутреннего использования).

В шапке сайта был добавлен компонент уведомлений в виде кнопки с бейджем, на котором выводится количество непрочитанных уведомлений. При нажатии на кнопку открывается выпадающий список и загружаются все уведомления пользователя. Реализована функция периодической загрузки количества непрочитанных уведомлений для отображения бейджа. В профиле пользователя, а также на всех прочих страницах, кроме страниц авторизации, также реализован блок для просмотра всех уведомлений с возможностью отметки о прочтении. Можно отметить прочитанными как единичные сообщения, так и все сразу.

Есть три типа поступающих сообщений:

- new_booking: создание бронирования, текст сообщения "Бронирование подтверждено";
- booking_rescheduled: перенос администратором, текст сообщения "Бронирование перенесено";
- booking_cancelled: Отмена администратором, текст сообщения "Бронирование отменено".

2.12.3 Результаты внедрения

После реализации данного решения были получены следующие результаты:

- уведомления о создании, переносе и отмене бронирования поступают автоматически;
- за отображение количества непрочитанных уведомлений отвечает бейдж с числом
- отметка о прочтении доступна и позволяет отметить одно либо все уведомления по желанию пользователя;
- происходит периодическое обновление каждые 5 секунд, чтобы пользователь всегда имел актуальные и своевременные уведомления;
- выпадающий список для вывода уведомлений реализован.

2.13 Завершение интеграции с базой данных

2.13.1 Постановка задачи

В процессе разработки информационной системы музыкальной студии была создана база данных, содержащая множество таблиц для хранения различных сущностей. Однако на момент начала работ небольшая часть таблиц оставалась не задействованной в бизнес-логике приложения. Требовалось завершить интеграцию всех таблиц базы данных, обеспечив их полноценное использование в работе системы, а также дописать недостающую логику для ранее упомянутых дополнительных инструментов и счетов.

2.13.2 Реализованное решение

Далее приведена структура БД и статусы использования таблиц в информационной системе до момента реализации решения:

- UsersAdvanced (пользователи системы) - используется;
- Equipment (инструменты для аренды) - используется;
- Rooms (помещения для аренды) - используется;
- RoomTypes (типы помещений) - используется;
- BookingsAdvanced (бронирования) - используется;
- BookingEquipments (связь бронирований с оборудованием) - не использовалась;
- BillsAdvanced (счета на оплату) - не использовалась;
- BillItemsAdvanced (позиции счетов) - не использовалась;
- SubscriptionPlans (тарифы абонементов) - частично;
- UserSubscriptionsAdvanced (абонементы пользователей) - частично;

- StaffAdvanced (сотрудники) - частично;
- Notifications (уведомления) - не использовалась;
- RoomEquipment (инструменты, которые есть в наличии в бронируемом помещении) - не использовалась.

Таблица StaffAdvanced в полной мере оказалась внедрена в систему во время разработки логики для роли администратора (пункты 2.4, 2.7). Внедрение BillsAdvanced и BillItemsAdvanced описано в пункте 2.8 документа. Описание реализации решения, задействующего таблицы BookingEquipments и RoomEquipment, дано в пункте 2.9 отчёта. SubscriptionPlans и UserSubscriptionsAdvanced участвуют в решении задачи с внедрением абонементов (пункт 2.11). Создание решения задачи с уведомления, включающего новую таблицу Notifications, описано в 2.12.

2.13.3 Результаты внедрения

В результате проведённой интеграции:

- задействованы все таблицы базы данных — каждая таблица выполняет свою функцию в бизнес-логике системы;
- обеспечена согласованность данных — все операции выполняются в рамках транзакций с сохранением ссылочной целостности;
- реализована полная финансовая отчётность — каждое бронирование сопровождается созданием счёта с детализацией позиций;
- внедрена система уведомлений — пользователь информируется о всех важных событиях;
- автоматизировано применение скидок — абонементы реально влияют на стоимость бронирования.

2.14 Улучшение пользовательского интерфейса

2.14.1 Постановка задачи

При выполнении всех предыдущих задач зачастую появлялись наброски, не всегда согласованные со стилем страницы, подразумевающим использование чёрного и белого как основных цветов, а также их оттенков. Данный минимализм призван снизить визуальную нагрузку на пользователя, сфокусировать внимание на контенте и придать интерфейсу современный и профессиональный вид. Следовательно, потребовалось переработать новые визуальные решения и скорректировать старые под монохромный стиль.

2.14.2 Реализованное решение

На новых страницах задан единый шрифт, используемый на предыдущих страницах (Inter), акцентный цвет текста в большинстве случаев чёрный. Тёмный и светлый оттенки серого используются как вспомогательные цвета для менее важной информации.

Для кнопок также реализован один и тот же стиль (чёрная прямоугольная кнопка с закруглением углов). Для улучшения тактильной обратной связи прописаны стили при наведении пользователем курсора на кнопку в зависимости от того, какое действие она позволяет выполнить: красный цвет текста и границы, если действие потенциально опасное для данных пользователя, и чёрный цвет текста и границы на белом фоне, если нет.

Цветные эмодзи на странице контактов заменены на нейтральные их аналоги. Логотипы социальных ресурсов (VK, YouTube, GitHub) также изначально были выполнены как цветные эмодзи, заменены на чёрно-белые логотипы реальных ресурсов.

Все фильтры на страницах каталогов выровнены по ширине их контейнера. Плавный переход, подобный ранее упомянутому поведению для кнопок, был реализован при наведении на карточки в каталогах помещений и инструментов: выбранная карточка выделяется тенью и слегка приподнимается, чтобы пользователь точно понимал, какую карточку просматривает.

2.14.3 Результаты внедрения

Проведённый редизайн пользовательского интерфейса позволил:

- создать единую визуальную идентичность — все страницы приложения выдержаны в едином стиле;
- повысить профессионализм восприятия — монохромная гамма ассоциируется с надёжностью и качеством;
- улучшить пользовательский опыт — последовательное поведение элементов при наведении и взаимодействии;
- снизить когнитивную нагрузку — пользователь не отвлекается на избыточные цветовые акценты;
- обеспечить лучшую доступность — монохромный интерфейс более удобен для людей с различными формами дальтонизма.

Заключение

В результате прохождения производственной (технологической/проектно-технологической) практики была продолжена разработка веб-приложения - информационной системы для управления деятельностью музыкальной студии. В ходе работы над проектом был реализован ряд дополнительных функций, значительно расширяющих базовый функционал системы.

Существующая архитектура клиент-серверного веб-приложения была доработана в соответствии с требованиями для внедрения новых функций. Реляционная база данных PostgreSQL содержит полный набор таблиц, охватывающих все бизнес-сущности и полностью задействованных в информационной системе. Практическая реализация проекта позволила закрепить навыки работы с языком JavaScript, средствами HTML и CSS, а также навыки интеграции фронтенд-части с серверной платформой ASP.NET.

За период практики проект значительно был улучшен как на уровне серверной логики, так и на уровне пользовательского интерфейса. Разработанное веб-приложение на данный момент охватывает ключевые бизнес-процессы: управление ресурсами, бронирование, учёт абонементов, финансовый учёт и уведомление пользователей. Есть ещё немалое количество возможных нововведений, которые можно добавить в дальнейшем, вроде уведомлений в браузере, интеграции с системами оплаты и календарём, панели аналитики и т.д., которые могут улучшить опыт работы с данной информационной системой и сделать её ещё более конкурентоспособной на фоне имеющихся аналогов. Эти предложения для следующих версий системы в дальнейшем будут мной рассмотрены и обдуманы.

Таким образом, цели и задачи производственной практики были достигнуты в полном объёме, а полученные в ходе работы знания и практические навыки могут быть применены в дальнейшей профессиональной деятельности в области разработки веб-приложений.

Список литературы

1. Документация по продуктам Microsoft: [сайт]. — URL: <https://learn.microsoft.com/ru-ru/> (дата обращения: 09.04.2026).
2. Документация PostgreSQL: [сайт]. — URL: <https://www.postgresql.org/docs/> (дата обращения: 18.04.2026).
3. Metanit.com: руководство по Entity Framework Core: [сайт]. — URL: <https://metanit.com/sharp/efcore/> (дата обращения: 08.10.2025).
4. MDN Web Docs: [сайт]. — URL: <https://developer.mozilla.org/en-US/> (дата обращения: 11.05.2026).
5. Документация Visual Studio Code: [сайт]. — URL: <https://code.visualstudio.com/docs> (дата обращения: 11.05.2026).
6. Репозиторий проекта на GitHub // GitHub: [сайт]. — URL: <https://github.com/limdizz/norbit-practice-project> (дата обращения: 22.05.2026).